

SHELL PROGRAMMING :GREATEST AMONG THREE NUMBERS

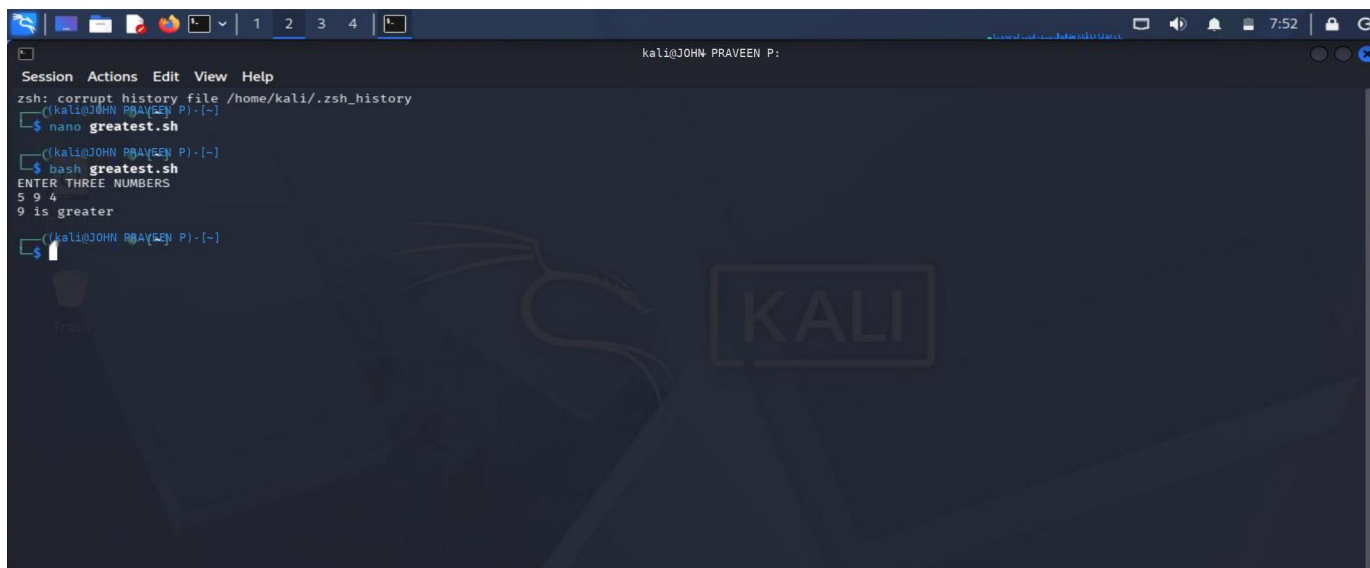


The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'greatest.sh'. The script content is as follows:

```
GNU nano 9.0 greatest.sh
#!/bin/bash
echo "ENTER THREE NUMBERS"
read a b c
if [ $a -gt $b ] && [ $a -gt $c ]
then
    echo "$a is greater"
elif [ $b -gt $c ]
then
    echo "$b is greater"
else
    echo "$c is greater"
fi
```

The terminal window title is 'kali@JOHN PRAVEEN P:'. The bottom status bar shows various keyboard shortcuts for nano editor operations.

OUTPUT :

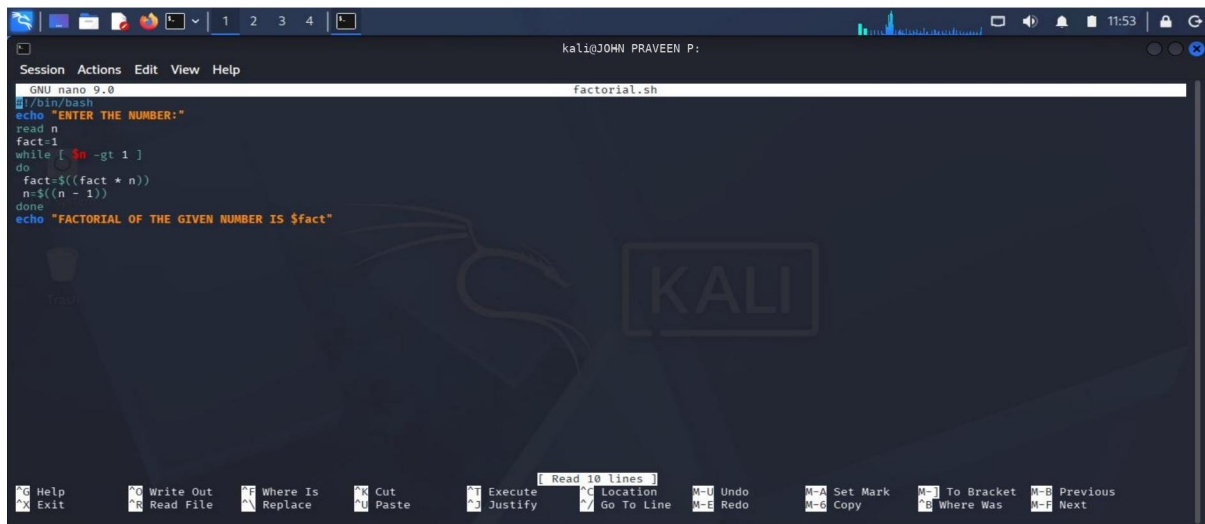


The screenshot shows the same terminal window after running the script. The output is as follows:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHN PRAVEEN P):-[~]
$ nano greatest.sh
(kali@JOHN PRAVEEN P):-[~]
$ bash greatest.sh
ENTER THREE NUMBERS
5 9 4
9 is greater
(kali@JOHN PRAVEEN P):-[~]
$
```

The terminal window title remains 'kali@JOHN PRAVEEN P:'. The bottom status bar shows the same keyboard shortcuts.

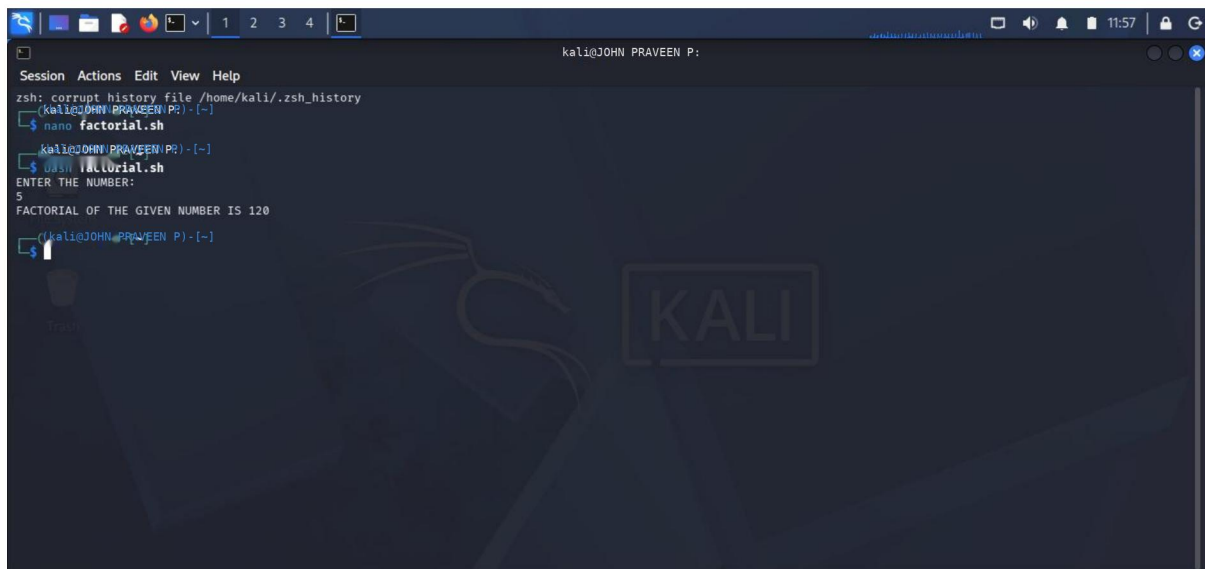
SHELL PROGRAMMING :FACTORIAL OF A GIVEN NUMBER



```
GNU nano 9.0 factorial.sh
#!/bin/bash
echo "ENTER THE NUMBER:"
read n
fact=1
while [ $n -gt 1 ]
do
    fact=$((fact * n))
    n=$((n - 1))
done
echo "FACTORIAL OF THE GIVEN NUMBER IS $fact"
```

The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'factorial.sh'. The script contains a shebang line, a prompt for the user to enter a number, a loop to calculate the factorial, and a final echo statement. The terminal title bar shows 'kali@JOHN PRAVEEN P:' and the time is 11:53.

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
kali@JOHN PRAVEEN P: ~$ nano factorial.sh
kali@JOHN PRAVEEN P: ~$ ./factorial.sh
ENTER THE NUMBER:
5
FACTORIAL OF THE GIVEN NUMBER IS 120
kali@JOHN PRAVEEN P: ~$
```

The screenshot shows the terminal output of the script. The user has run the script, entered the number 5, and the script has outputted the factorial of 5, which is 120. The terminal title bar shows 'kali@JOHN PRAVEEN P:' and the time is 11:57.

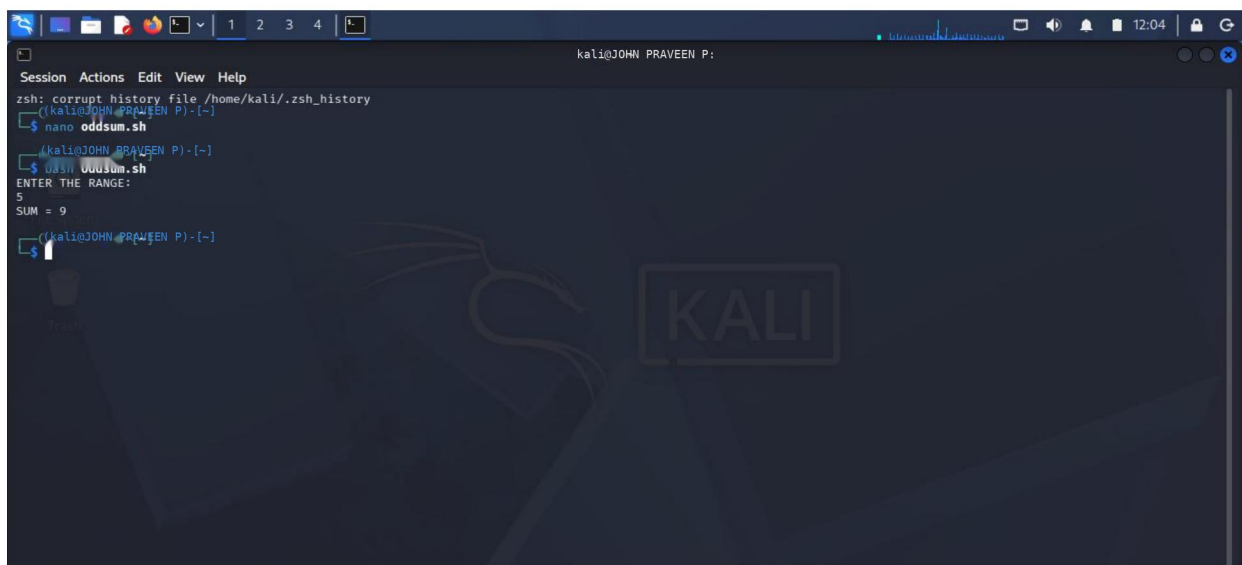
SHELL PROGRAMMING :SUM OF ODD NUMBERS



```
GNU nano 9.0 oddsum.sh
#!/bin/bash
echo "ENTER THE RANGE:"
read n
sum=0
x=1
while [ $x -le $n ]
do
    sum=$((sum + x))
    x=$((x + 2))
done
echo "SUM = $sum"
```

The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'oddsum.sh'. The script contains a shebang line, a prompt for the user to enter a range, a loop that calculates the sum of odd numbers from 1 to the entered range, and an echo statement to display the result. The terminal title bar shows 'kali@JOHN PRAVEEN P:' and the system clock is 12:01.

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHN PRAVEEN P)-[~]
$ nano oddsum.sh
(kali@JOHN PRAVEEN P)-[~]
$ ./oddsum.sh
ENTER THE RANGE:
5
SUM = 9
(kali@JOHN PRAVEEN P)-[~]
$
```

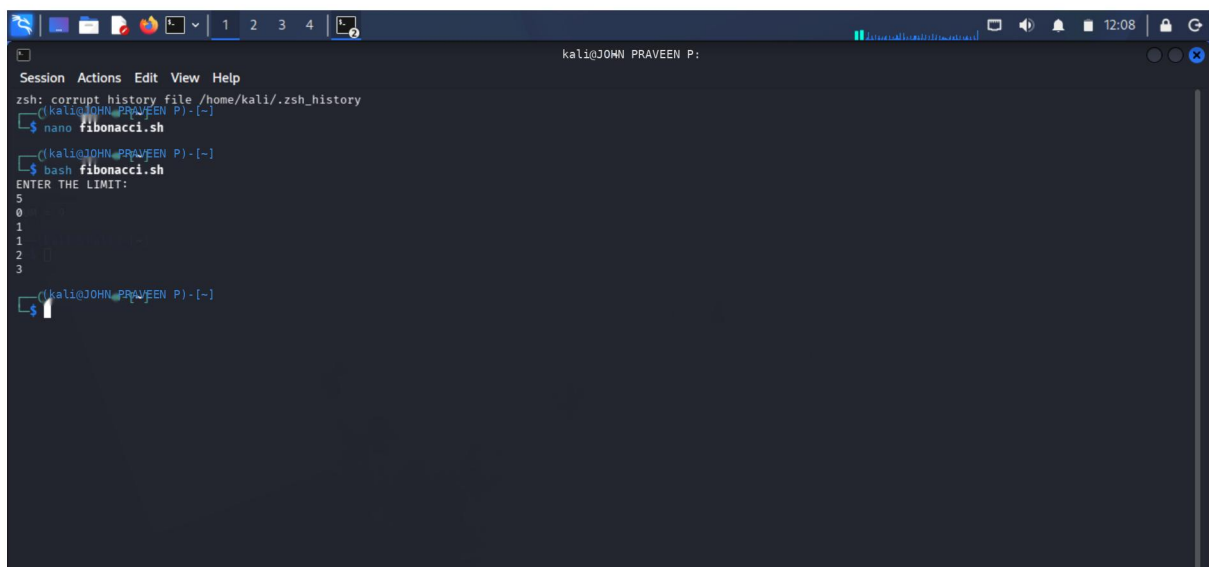
The screenshot shows the terminal output of the script. The user has entered '5' as the range, and the script has calculated the sum of odd numbers from 1 to 5, which is 9. The terminal title bar shows 'kali@JOHN PRAVEEN P:' and the system clock is 12:04.

SHELL PROGRAMMING :FIBONACCI NUMBER



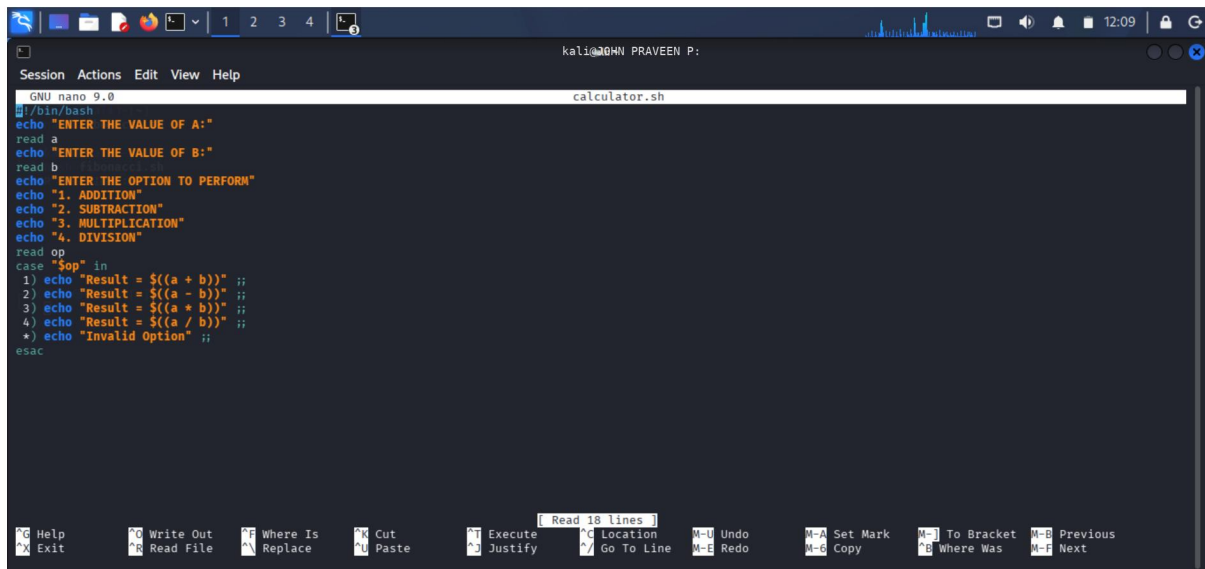
```
GNU nano 9.0 fibonacci.sh
#!/bin/bash
echo "ENTER THE LIMIT:"
read n
p=1
q=1
i=1
while [ $i -le $n ]
do
r=$((p + q))
p=$q
q=$r
echo "$r"
i=$((i + 1))
done
```

OUTPUT :



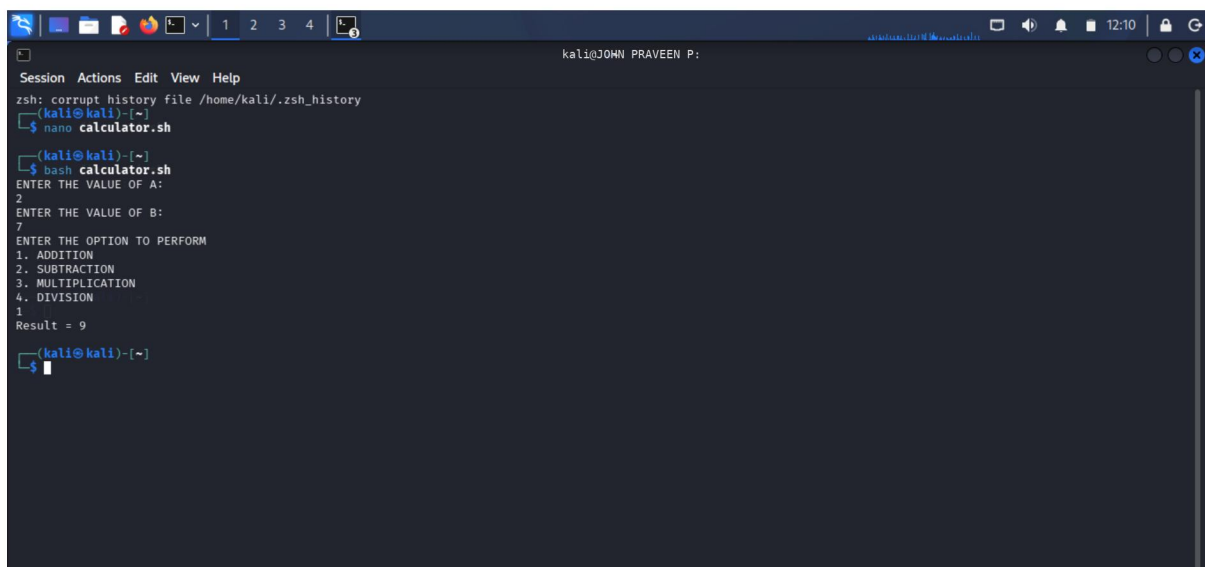
```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHN PRAVEEN P: ~)
$ nano fibonacci.sh
(kali@JOHN PRAVEEN P: ~)
$ bash fibonacci.sh
ENTER THE LIMIT:
5
0
1
1
2
3
(kali@JOHN PRAVEEN P: ~)
$
```

SHELL PROGRAMMING :ARITHMETIC CALCULATOR



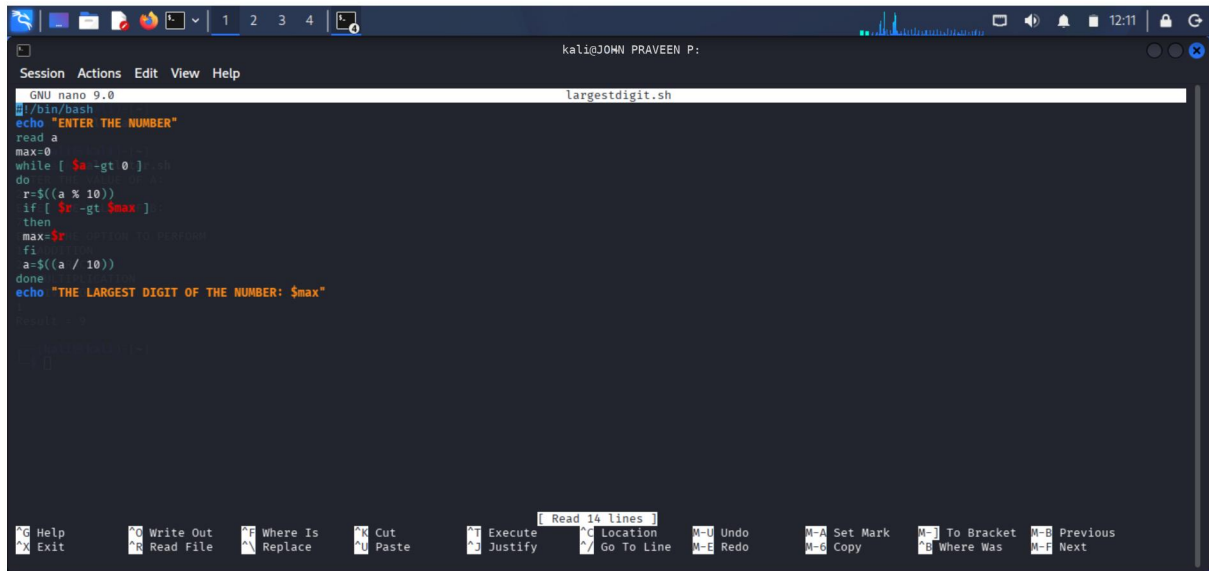
```
GNU nano 9.0 calculator.sh
#!/bin/bash
echo "ENTER THE VALUE OF A:"
read a
echo "ENTER THE VALUE OF B:"
read b
echo "ENTER THE OPTION TO PERFORM"
echo "1. ADDITION"
echo "2. SUBTRACTION"
echo "3. MULTIPLICATION"
echo "4. DIVISION"
read op
case "$op" in
1) echo "Result =  $$(a + b)$ " ;;
2) echo "Result =  $$(a - b)$ " ;;
3) echo "Result =  $$(a * b)$ " ;;
4) echo "Result =  $$(a / b)$ " ;;
*) echo "Invalid Option" ;;
esac
```

OUTPUT :



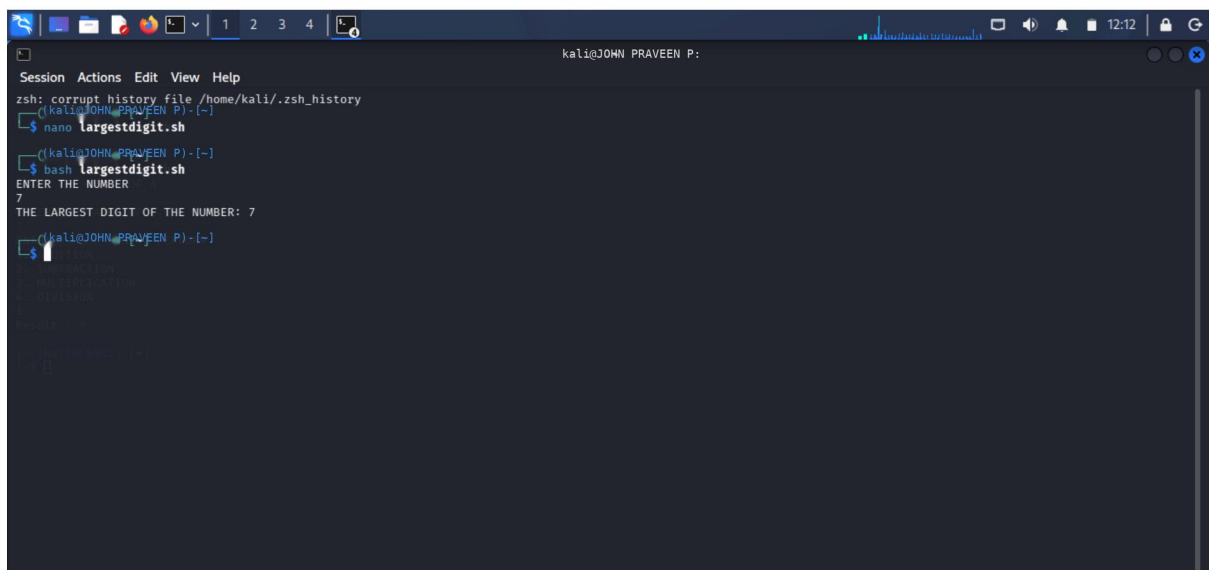
```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$ nano calculator.sh
(kali@kali)~$ bash calculator.sh
ENTER THE VALUE OF A:
2
ENTER THE VALUE OF B:
7
ENTER THE OPTION TO PERFORM
1. ADDITION
2. SUBTRACTION
3. MULTIPLICATION
4. DIVISION
1
Result = 9
(kali@kali)~$
```

SHELL PROGRAMMING :LARGEST DIGIT OF A NUMBER



```
GNU nano 9.0 largestdigit.sh
#!/bin/bash
echo "ENTER THE NUMBER"
read a
max=0
while [ $a -gt 0 ]
do
r=$((a % 10))
if [ $r -gt $max ]
then
max=$r
fi
a=$((a / 10))
done
echo "THE LARGEST DIGIT OF THE NUMBER: $max"
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHN PRAVEEN P: ~)
$ nano largestdigit.sh
(kali@JOHN PRAVEEN P: ~)
$ bash largestdigit.sh
ENTER THE NUMBER
7
THE LARGEST DIGIT OF THE NUMBER: 7
(kali@JOHN PRAVEEN P: ~)
$
```

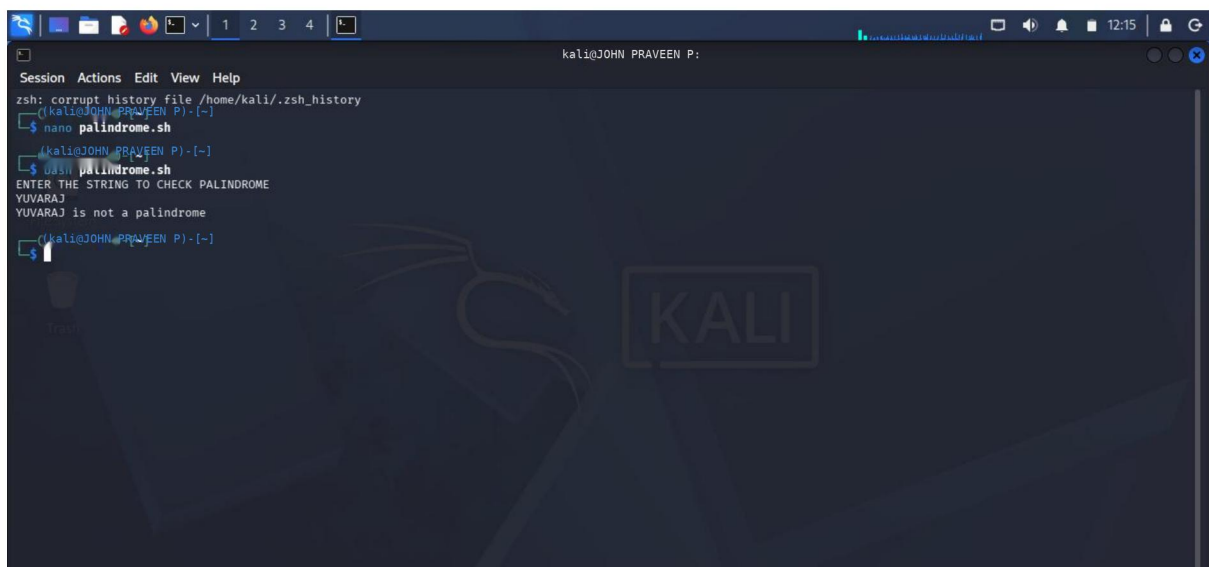
SHELL PROGRAMMING :PALINDROME OR NOT



The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'palindrome.sh'. The script is written in bash and checks if a string is a palindrome. It prompts the user to enter a string, reads it, and then uses a while loop and string manipulation to compare characters from both ends of the string. The script outputs a message indicating whether the string is a palindrome or not.

```
GNU nano 9.0
#!/bin/bash
echo "ENTER THE STRING TO CHECK PALINDROME"
read str
len=$(echo -n "$str" | wc -c)
i=1
j=$((len / 2))
while [ $i -le $j ]
do
k=$(echo "$str" | cut -c $i)
l=$(echo "$str" | cut -c $len)
if [ "$k" != "$l" ]
then
echo "$str is not a palindrome"
exit
fi
i=$((i + 1))
len=$((len - 1))
done
echo "$str is a palindrome"
```

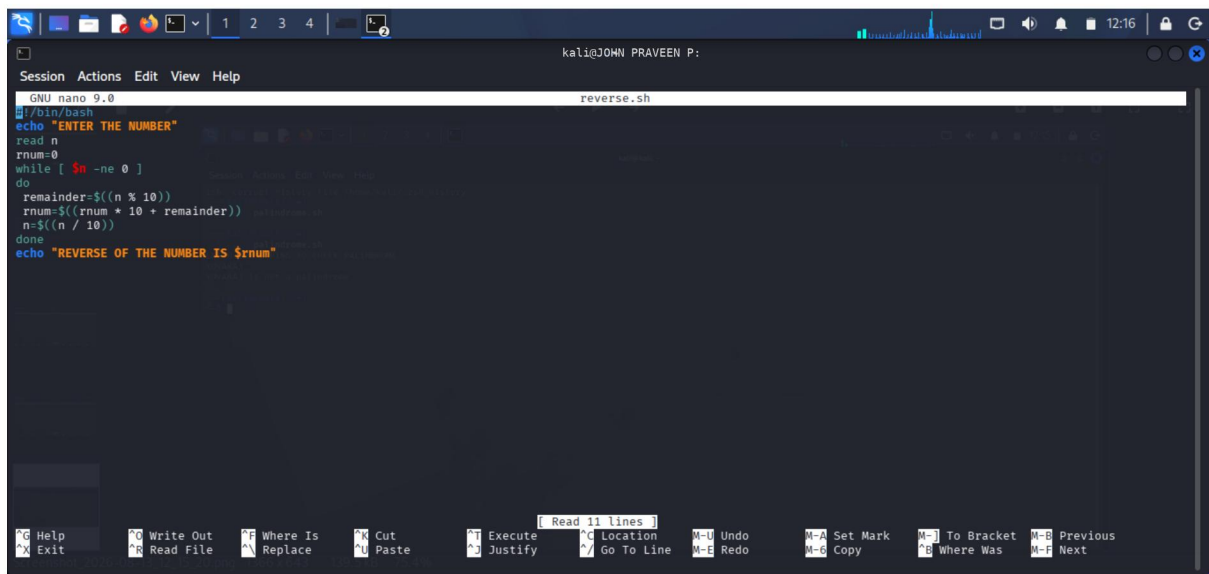
OUTPUT :



The screenshot shows a terminal window with the following commands and output:

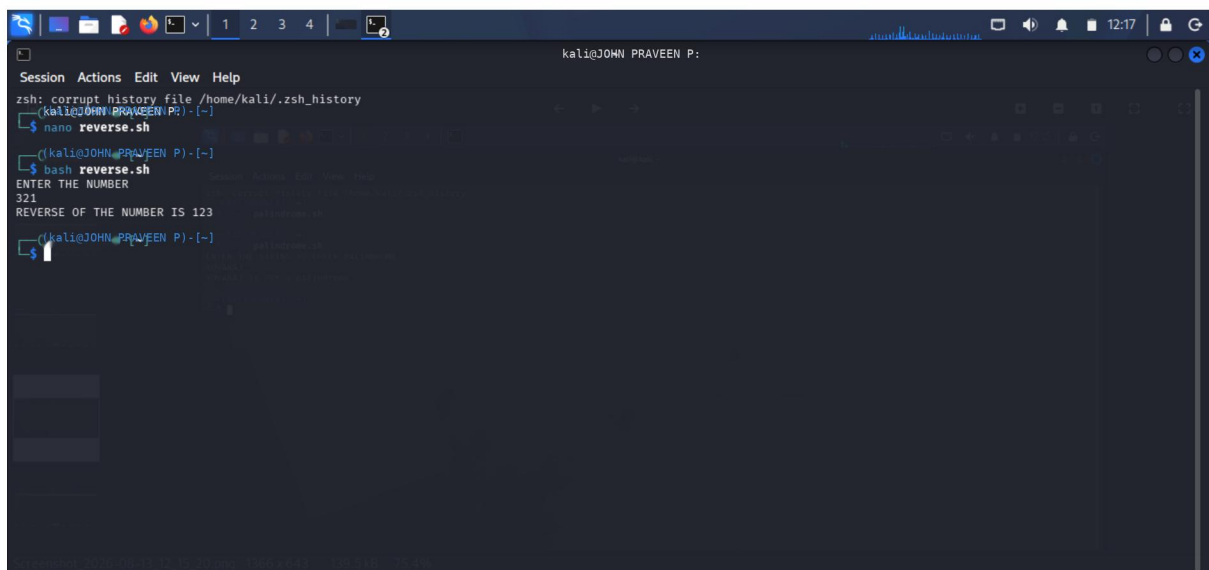
```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHN PRAVEEN P:~)
$ nano palindrome.sh
(kali@JOHN PRAVEEN P:~)
$ bash palindrome.sh
ENTER THE STRING TO CHECK PALINDROME
YUVARAJ
YUVARAJ is not a palindrome
(kali@JOHN PRAVEEN P:~)
$
```

SHELL PROGRAMMING :REVERSE OF A GIVEN NUMBER



```
GNU nano 9.0 reverse.sh
#!/bin/bash
echo "ENTER THE NUMBER"
read n
rnum=0
while [ $n -ne 0 ]
do
    remainder=$((n % 10))
    rnum=$((rnum * 10 + remainder))
    n=$((n / 10))
done
echo "REVERSE OF THE NUMBER IS $rnum"
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHN PRAVEEN P: ~)
$ nano reverse.sh
(kali@JOHN PRAVEEN P: ~)
$ bash reverse.sh
ENTER THE NUMBER
321
REVERSE OF THE NUMBER IS 123
(kali@JOHN PRAVEEN P: ~)
$
```